Practical – 9

Aim-1 :

Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

Code :

```
class Counter {
    int count = 0;

    // Without synchronization
    void increment() {
        count++;
    }

    // With synchronization
    synchronized void incrementSynchronized() {
        count++;
    }
}

public class CounterRace {

    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        Counter counter = new Counter();

        int numberOfThreads = 10;
        int incrementsPerThread = 1000;

        Thread[] threads = new Thread[numberOfThreads];

        // Create threads
        for (int i = 0; i < numberOfThreads; i++) {

            threads[i] = new Thread(() -> {

                for (int j = 0; j < incrementsPerThread; j++) {
                    counter.increment();
                }
```

```
            });
        }

        // Start threads
        for (Thread thread : threads) {
            thread.start();
        }

        // Wait for all threads
        for (Thread thread : threads) {
            thread.join();
        }

        int expected = numberOfThreads * incrementsPerThread;

        System.out.println("Expected count: " + expected);
        System.out.println("Actual count: " + counter.count);
    }
}
```

Output :

```
Expected count: 10000
Actual count: 9422


...Program finished with exit code 0
Press ENTER to exit console.
```

Learning Outcomes:

After completing this program, students will be able to:

1. **Understand multithreading** by creating and running multiple threads.
2. **Understand race conditions** when multiple threads access shared data.
3. **Use the `Thread` class** to create and execute threads.
4. **Use `start()` and `join()`** for thread execution and synchronization.
5. **Understand synchronization** using the `synchronized` keyword.
6. **Compare synchronized and unsynchronized methods** when accessing shared variables.
7. **Understand why the actual count may differ from the expected count** due to concurrent access.

Aim-2 :

Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.

Code :



```
class SeatManager {

    private int seatsLeft = 5;

    // Without synchronization
    boolean bookSeatUnsafe() {

        if (seatsLeft > 0) {

            // Artificial delay to increase race condition
            try {
                Thread.sleep(millis: 10);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            seatsLeft--;

            return true;
        }

        return false;
    }

    // With synchronization
    synchronized boolean bookSeatSafe() {

        if (seatsLeft > 0) {

            seatsLeft--;

            return true;
        }
```

CEUE20325IT103

```
        }

        return false;
    }

    int getSeatsLeft() {
        return seatsLeft;
    }
}

public class SeatBooking {

    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        SeatManager manager = new SeatManager();

        int numberOfThreads = 10;

        Thread[] threads = new Thread[numberOfThreads];

        for (int i = 0; i < numberOfThreads; i++) {

            final int threadNumber = i + 1;

            threads[i] = new Thread(() -> {

                boolean booked = manager.bookSeatSafe();

                if (booked) {
                    System.out.println(
```

```
                boolean booked = manager.bookSeatSafe();

                if (booked) {
                    System.out.println(
                        "Thread " + threadNumber + " successfully booked a seat."
                    );
                } else {
                    System.out.println(
                        "Thread " + threadNumber + " failed to book a seat."
                    );
                }

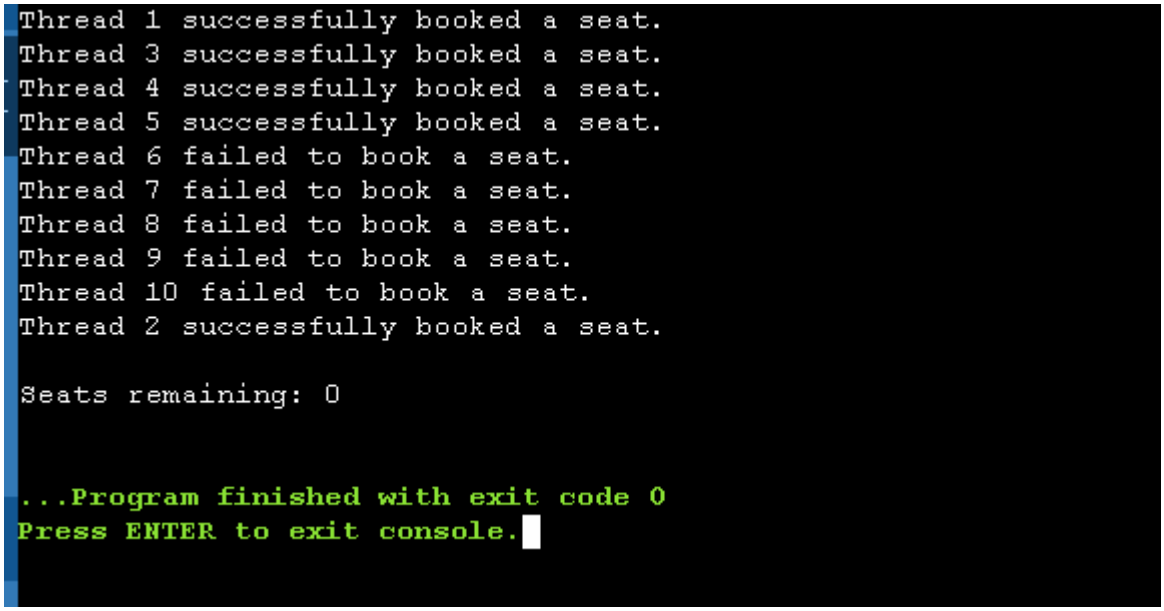
            });
        }

        for (Thread thread : threads) {
            thread.start();
        }

        for (Thread thread : threads) {
            thread.join();
        }

        System.out.println();
        System.out.println("Seats remaining: " + manager.getSeatsLeft());
    }
}
```

Output:



Learning Outcomes:

After completing this **Seat Booking and Thread Synchronization** program, students will be able to:

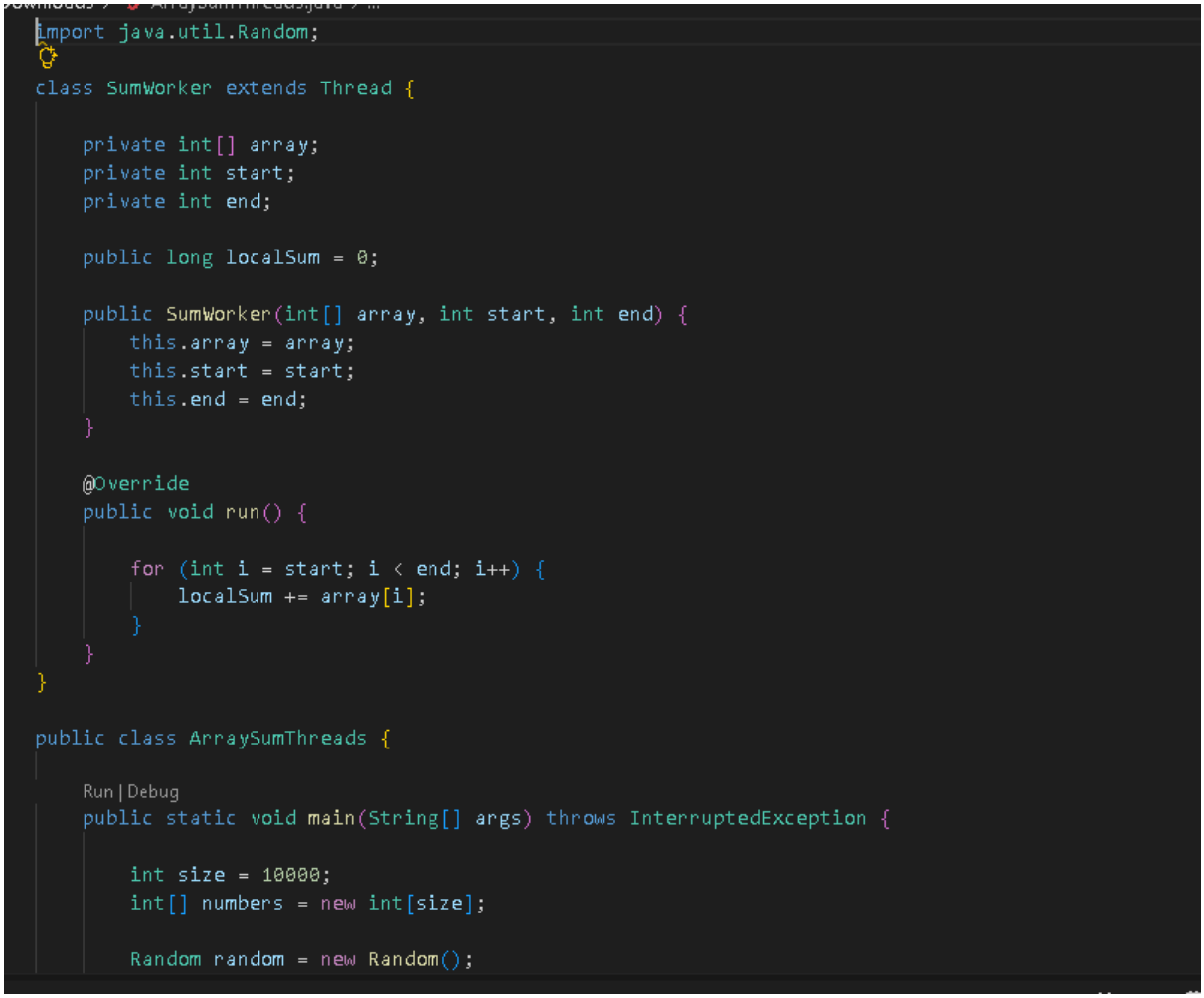
1. **Understand multithreading** by creating multiple threads for simultaneous seat booking.
2. **Understand race conditions** when multiple threads access shared seat data.
3. **Use the `synchronized` keyword** to safely manage shared resources.
4. **Understand thread-safe operations** in a concurrent program.
5. **Use `start()` and `join()`** to control thread execution.
6. **Handle `InterruptedException`** using `try-catch`.
7. **Compare safe and unsafe resource access** in multithreaded applications.
8. **Implement a real-world concurrent application** such as a seat booking system.

Aim-3 :

(For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference.

Code:

CEUE20325IT103



```java
import java.util.Random;

class SumWorker extends Thread {

    private int[] array;
    private int start;
    private int end;

    public long localSum = 0;

    public SumWorker(int[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    public void run() {

        for (int i = start; i < end; i++) {
            localSum += array[i];
        }
    }
}

public class ArraySumThreads {

    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        int size = 10000;
        int[] numbers = new int[size];

        Random random = new Random();
```

CEUE20325IT103

```
        Random random = new Random();

        // Generate random numbers
        for (int i = 0; i < size; i++) {
            numbers[i] = random.nextInt(bound: 100);
        }

        int numberOfThreads = 4;

        SumWorker[] workers = new SumWorker[numberOfThreads];

        int partSize = size / numberOfThreads;

        // Create workers
        for (int i = 0; i < numberOfThreads; i++) {

            int start = i * partSize;

            int end;

            if (i == numberOfThreads - 1) {
                end = size;
            } else {
                end = start + partSize;
            }

            workers[i] = new SumWorker(numbers, start, end);
        }

        // Start threads
        for (SumWorker worker : workers) {
```

```java
        // Start threads
        for (SumWorker worker : workers) {
            worker.start();
        }

        // Wait for threads
        for (SumWorker worker : workers) {
            worker.join();
        }

        // Combine results
        long total = 0;

        for (SumWorker worker : workers) {
            total += worker.localSum;
        }

        System.out.println("Array size: " + size);
        System.out.println("Number of threads: " + numberOfThreads);
        System.out.println("Total sum: " + total);
    }
}
```

Output :

```
Array size: 10000
Number of threads: 4
Total sum: 494481


...Program finished with exit code 0
Press ENTER to exit console.
```

Learning Outcomes:

After completing this **Array Sum Using Multiple Threads** program, students will be able to:

1. **Understand multithreading** by creating multiple worker threads.
2. **Divide an array into smaller parts** and assign each part to a separate thread.
3. **Extend the `Thread` class** and override the `run()` method.
4. **Use `start()` and `join()`** for thread execution and synchronization.
5. **Calculate partial sums independently** using multiple threads.
6. **Combine the results** from all threads to calculate the total sum.
7. **Understand parallel processing** and how a large task can be divided into smaller tasks.

8. **Use `Random` class** to generate random array values.